

Backbone Curve & Bilinear Idealization Tool

A Comprehensive Cyclic Hysteresis Analysis Suite

ASTM E2126 EEEP Method Implementation

Graphical User Interface Application

Developed By

Tufail Mabood

UET Peshawar

Repository: github.com/tufailmab/ASTM-E2126-Backbone-Curve-Tool

DOI: [10.5281/zenodo.21805267](https://doi.org/10.5281/zenodo.21805267)

License: MIT License

Version 1.0

August 2026

Open-source Python application for structural engineering analysis

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	The ASTM E2126 EEEP Method	5
1.3	Scope and Objectives	5
1.4	Key Features	5
2	Theoretical Background	6
2.1	Structural Dynamics and Hysteresis	6
2.2	Hysteresis Loop Characteristics	7
2.3	Backbone Curve Theory	7
2.4	Monotonic Envelope Extraction	8
2.5	EEEE Method Formulation	8
2.5.1	Initial Stiffness	9
2.5.2	Ultimate Point	9
2.5.3	Yield Point	9
2.5.4	Ductility	10
2.6	Energy Dissipation	10
2.7	Stiffness Degradation	11
3	Software Architecture and Design	11
3.1	System Architecture	11
3.2	Class Structure	11
3.2.1	Data Management	12
3.2.2	Processing Pipeline	12
3.3	Dependency Management	12
3.4	Data Flow Diagram	13
3.5	Threading Model	13
4	Mathematical Algorithms and Implementations	14
4.1	Data Loading and Validation	14
4.1.1	File Parsing	14
4.1.2	Data Validation	15
4.2	Reversal Point Detection	15
4.2.1	Gradient Calculation	15
4.2.2	Sign Determination	15
4.2.3	Turning Point Detection	16
4.3	Monotonic Envelope Extraction	16
4.3.1	Sorting by Absolute Displacement	16
4.3.2	Clustering	16
4.3.3	Cluster Reduction	17
4.3.4	Monotonicity Enforcement	17
4.4	Backbone Curve Construction	17
4.4.1	Branch Separation	17
4.4.2	Monotonic Envelope Extraction	18
4.4.3	Combined Backbone	18
4.5	Mirroring Negative Branch	18
4.5.1	Reversal and Sign Change	18
4.6	Average Branch Calculation	18
4.6.1	Common Grid	18

4.6.2	Interpolation	19
4.6.3	Averaging	19
4.7	Branch Property Computation	19
4.7.1	Peak Point	20
4.7.2	Ultimate Point	20
4.7.3	Initial Stiffness	20
4.7.4	Energy Dissipation	20
4.7.5	EEEP Bilinear Idealization	20
4.8	Cycle Identification	21
4.8.1	Positive Peak Detection	21
4.8.2	Cycle Segmentation	21
4.8.3	Peak Extraction	21
4.9	Cycle Metrics Computation	22
4.9.1	Secant Stiffness	22
4.9.2	Stiffness Retention	22
4.9.3	Dissipated Energy	22
4.9.4	Cumulative Energy	23
5	GUI Implementation and Features	23
5.1	Layout Structure	23
5.2	Interactive Plot Features	23
5.2.1	Mouse Interaction	23
5.2.2	Crosshair Cursor	23
5.2.3	Data Inspection	24
5.3	Engineering Information Panel	24
5.4	Processing Log Implementation	25
5.5	Summary Table Implementation	25
5.5.1	Sorting	25
5.5.2	Filtering	26
5.5.3	Highlighting	26
6	Output Generation and File Formats	26
6.1	Directory Structure	26
6.2	CSV File Formats	26
6.2.1	Backbone Curve CSV	27
6.2.2	Positive/Negative/Average CSV	27
6.2.3	Bilinear Idealization CSV	27
6.2.4	Cyclic Properties CSV	27
6.3	Plot Generation	27
6.3.1	Backbone Curve Plot	28
6.3.2	Bilinear Idealization Plot	28
6.3.3	Stiffness Degradation Plot	28
6.3.4	Energy Dissipation Plot	28
6.4	Master Summary CSV	28
7	User Preferences and Persistence	29
7.1	Preferences Storage	29
7.2	Preference Categories	29
7.3	Preference JSON Schema	29
7.4	Keyboard Shortcuts	30

8	Complete Equations Reference	30
8.1	Data Processing Equations	30
8.2	Envelope Extraction Equations	31
8.3	Backbone Curve Equations	31
8.4	Mirroring Equations	31
8.5	Average Branch Equations	32
8.6	Peak Point Equations	32
8.7	Ultimate Point Equations	32
8.8	Stiffness Equations	32
8.9	Energy Equations	33
8.10	EEEP Bilinear Idealization Equations	33
8.11	Cycle Identification Equations	33
8.12	Cycle Metrics Equations	33
8.13	Plot Generation Equations	34
9	Usage Guide	34
9.1	Installation	34
9.1.1	Python Installation	34
9.1.2	Download Application	34
9.1.3	Run Application	34
9.2	Data Preparation	34
9.2.1	File Format	34
9.2.2	Example Input	35
9.3	Processing Workflow	35
9.3.1	Step 1: Select Input Folder	35
9.3.2	Step 2: Select Output Folder	35
9.3.3	Step 3: Process Files	36
9.3.4	Step 4: Monitor Progress	36
9.4	Result Review	36
9.4.1	Plot Viewer	36
9.4.2	Engineering Information	36
9.4.3	Summary Table	36
9.5	Export Results	36
9.5.1	Export Plot	36
9.5.2	Copy Plot	37
9.5.3	Export Log	37
9.5.4	Export Summary	37
10	Conclusion	37
10.1	Summary	37
10.2	Applications	37
10.3	Future Developments	37
10.4	Final Remarks	38
A	Glossary of Technical Terms	38

List of Tables

1	Application Key Features	6
2	Hysteresis Loop Characteristics	7
3	EEEP Method Parameters	8
4	System Architecture Layers	11
5	Processing Pipeline Stages	12
6	Software Dependencies	13
7	Threading Model Components	14
8	Data Loading Algorithm Steps	14
9	Reversal Point Detection Algorithm	15
10	Monotonic Envelope Extraction Algorithm	16
11	Backbone Curve Construction Algorithm	17
12	Average Branch Calculation Algorithm	19
13	Branch Property Computation	19
14	Cycle Identification Algorithm	21
15	Cycle Metrics Formulas	22
16	Interactive Plot Features	24
17	Mouse Interaction Response	24
18	Information Panel Fields	24
19	Processing Log Color Coding	25
20	Summary Table Features	25
21	CSV File Formats	26
22	Plot Types and Styling	27
23	Summary CSV Columns	28
24	Preference Categories	29
25	Keyboard Shortcuts	30
26	Input File Format Specification	35
27	Processing Workflow Steps	35
28	Export Options Summary	36
29	Application Areas and Use Cases	38
30	Glossary of Technical Terms Used in This Document	38

1 Introduction

1.1 Background and Motivation

The analysis of cyclic hysteresis behaviour is fundamental to understanding the seismic performance of structural components subjected to earthquake loading. Reinforced concrete and steel elements, when exposed to cyclic loading, exhibit complex nonlinear responses that are characterized by several distinct phenomena. These include the formation of hysteresis loops which represent the energy dissipation capacity of the structure, progressive stiffness degradation with increasing number of cycles, strength degradation due to cumulative damage, and pinching effects which manifest as a narrowing of the hysteresis loops. The backbone curve, which represents the envelope of the hysteresis loops, is particularly important because it provides a simplified yet accurate representation of the structural behaviour that can be used for performance-based seismic design, capacity assessment of existing structures, calibration of numerical models, development of simplified analysis methods, and quality control in testing programs.

1.2 The ASTM E2126 EEEP Method

The Equivalent Energy Elastic-Plastic (EEEP) method, standardized in ASTM E2126, provides a systematic and rigorous approach for characterizing the cyclic behaviour of structural components. This method has gained widespread acceptance in the structural engineering community due to its theoretical foundation and practical applicability. The method involves several key steps: first, extracting the backbone curve from the cyclic response data; second, determining the initial stiffness K_e which represents the elastic behaviour; third, identifying the yield point (D_y, F_y) using equal energy principles that ensure the idealized model preserves the energy dissipation capacity; fourth, locating the ultimate point (D_u, F_u) which defines the deformation capacity; fifth, calculating the displacement ductility $\mu = D_u/D_y$ which is a measure of the structure's ability to deform beyond yielding; and finally, evaluating energy dissipation and stiffness degradation across loading cycles.

The fundamental principle of the EEEP method is that the area under the actual backbone curve up to the ultimate point must equal the area under the idealized elastic-perfectly-plastic curve, as expressed in Equation 1. This equal energy principle is of paramount importance because it ensures that the idealized model preserves the energy dissipation capacity of the actual system, which is crucial for seismic performance assessment.

$$\int_0^{D_u} F_{\text{actual}}(D) dD = \int_0^{D_u} F_{\text{idealized}}(D) dD \quad (1)$$

1.3 Scope and Objectives

This document provides comprehensive technical documentation for the **Backbone Curve & Bilinear Idealization Tool**, a sophisticated Python-based GUI application that implements the complete ASTM E2126 EEEP methodology. The primary objectives of this documentation are to describe the software architecture and design principles, document all mathematical formulations and algorithms in detail, provide comprehensive usage instructions for both novice and advanced users, explain the output generation and interpretation, serve as a reference for developers and users, and facilitate quality assurance and validation of results.

1.4 Key Features

The application provides a comprehensive set of professional-grade features that are summarized in Table 1. These features have been carefully designed to meet the needs of structural engineers, researchers, and educators working with cyclic test data.

Table 1: Application Key Features

Feature Category	Description
Batch Processing	Automatic processing of multiple specimens in a single operation, significantly reducing analysis time
Interactive Visualization	Professional-quality plots with zoom, pan, and data inspection capabilities
Comprehensive Analysis	Full implementation of the EEEP method including all ASTM E2126 requirements
Data Export	Multiple formats for plots (PNG, SVG, PDF) and tabular data (CSV)
User-friendly Interface	ETABS/SAP2000-style engineering workstation with intuitive navigation
Cross-platform	Works seamlessly on Windows, macOS, and Linux operating systems
Real-time Feedback	Live processing log with color-coded status for immediate error detection
Data Inspection	Click-to-inspect nearest data points with coordinate display
Persistent Settings	Remembers user preferences between sessions for enhanced productivity

2 Theoretical Background

2.1 Structural Dynamics and Hysteresis

The dynamic response of a structure under cyclic loading can be described by the well-known equation of motion presented in Equation 2. This second-order differential equation governs the behaviour of single-degree-of-freedom systems and forms the basis for understanding structural response under dynamic loading conditions.

$$M\ddot{u}(t) + C\dot{u}(t) + R(u(t)) = F(t) \quad (2)$$

In this equation, the mass matrix M represents the inertial properties of the structure, the damping matrix C accounts for energy dissipation mechanisms, the restoring force function $R(u)$ describes the nonlinear force-displacement relationship, and the applied load $F(t)$ represents the external forcing function. The displacement response $u(t)$ is the unknown quantity to be determined.

The restoring force function $R(u)$ is inherently nonlinear and exhibits hysteresis, meaning that the force-displacement relationship depends on the entire loading history rather than just the current displacement. This path-dependent behaviour is captured by Equation 3, which decomposes the restoring force into elastic and plastic components.

$$R(u) = R_{el}(u) + R_{pl}(u, \text{history}) \quad (3)$$

The elastic component $R_{el}(u)$ represents the recoverable deformation that disappears upon unloading, while the plastic component $R_{pl}(u, \text{history})$ represents the permanent deformation that accumulates progressively with each loading cycle. This decomposition is fundamental to understanding the cyclic behaviour of structures and forms the basis for many simplified analysis methods.

2.2 Hysteresis Loop Characteristics

A typical hysteresis loop exhibits several key characteristics that are essential for understanding structural behaviour under cyclic loading. These characteristics are summarized in Table 2, which provides a comprehensive overview of the phenomena observed in cyclic testing.

Table 2: Hysteresis Loop Characteristics

Characteristic	Description
Elastic loading	Initial linear response characterized by the initial stiffness K_e , representing the undamaged state
Yielding	Onset of nonlinear behaviour at the yield point (D_y, F_y) , marking the transition from elastic to plastic response
Plastic deformation	Permanent deformation that remains after unloading, representing accumulated damage
Energy dissipation	Area enclosed by the loop, calculated as $E = \oint F dD$, representing the energy absorbed per cycle
Stiffness degradation	Progressive reduction in slope with increasing cycles, reflecting cumulative damage
Bauschinger effect	Strength reduction in reverse loading due to the reorientation of internal stresses
Pinching	Narrowing of the loop due to crack opening and closing mechanisms
Strength degradation	Progressive reduction in peak force with increasing cycles

The energy dissipated in a single cycle is of particular importance for seismic analysis, as it represents the energy absorption capacity of the structure. This energy is given by the area enclosed by the hysteresis loop, as expressed in Equation 4. The integration is performed over a complete cycle, starting and ending at the same point on the loop.

$$E_{\text{cycle}} = \oint_{\text{cycle}} F(D) dD \quad (4)$$

2.3 Backbone Curve Theory

The backbone curve represents the envelope of the hysteresis loops and is defined as the maximum force attained at each displacement level. This concept is fundamental to the EEEP method and provides a simplified representation of the cyclic response. The formal definition of the backbone curve is given in Equation 5, which states that for each displacement, the backbone force is the maximum force achieved across all cycles.

$$F_{\text{backbone}}(D) = \max_{\text{cycles}} \{F_{\text{cycle}}(D)\} \quad (5)$$

More formally, for a set of cycles $C_1, C_2, \dots, C_n$, each with its own force-displacement relationship $F_i(D)$, the backbone curve is expressed in Equation 6. This mathematical formulation ensures that the backbone curve truly represents the outermost bounds of the cyclic response.

$$F_{\text{backbone}}(D) = \max\{F_i(D) \mid i = 1, \dots, n\} \quad (6)$$

The backbone curve typically consists of three distinct regions: the ascending branch from the origin to the peak point, which represents the initial loading path; the descending (post-peak) branch to the ultimate point, which represents the softening behaviour; and optionally, a residual

strength plateau at large deformations. Each of these regions provides important information about the structural behaviour.

2.4 Monotonic Envelope Extraction

The monotonic envelope extraction is a critical step in the backbone curve generation process. This procedure involves sorting the data points by absolute displacement and applying a sophisticated clustering algorithm to identify the outermost points of the response envelope. The clustering tolerance, defined in Equation 7, is set to 2% of the maximum absolute displacement, which has been found to provide optimal results in practice.

$$\tau = 0.02 \times \max(|D|) \quad (7)$$

For each cluster of points identified by the algorithm, the point with the maximum absolute force is retained, as shown in Equation 8. This approach ensures that the extracted envelope truly represents the maximum forces achieved at each displacement level.

$$(\bar{D}_j, \bar{F}_j) = \arg \max_{i \in \text{cluster}_j} |F_i| \quad (8)$$

The monotonic envelope is then constructed by retaining only those points where the absolute displacement strictly increases, as expressed in Equation 9. This monotonicity requirement is essential to ensure that the envelope represents a well-defined function, which is necessary for subsequent analysis steps.

$$\text{Envelope} = \{(\bar{D}_j, \bar{F}_j) \mid \bar{D}_j > \bar{D}_{j-1}\} \quad (9)$$

2.5 EEEP Method Formulation

The EEEP method is based on several key parameters that together characterize the cyclic behaviour of the structure. These parameters, which are defined in Table 3, form the basis for the bilinear idealization and provide a comprehensive description of the structural response.

Table 3: EEEP Method Parameters

Parameter	Symbol	Description
Peak Force	$F_{\max}$	Maximum force attained during the entire loading history
Peak Displacement	$D_{\max}$	Displacement at which the peak force is attained
Ultimate Force	F_u	Force at the ultimate point, defined as 80% of the peak force
Ultimate Displacement	D_u	Displacement at which the ultimate force is attained
Initial Stiffness	K_e	Secant stiffness through 40% of the peak force on the ascending branch
Yield Force	F_y	Force at the idealized yield point determined by the equal energy principle
Yield Displacement	D_y	Displacement at the idealized yield point
Ductility	μ	Ratio of ultimate displacement to yield displacement

2.5.1 Initial Stiffness

The initial stiffness K_e is a fundamental parameter that characterizes the elastic behaviour of the structure. It is defined as the secant stiffness through 40% of the peak force on the ascending branch of the backbone curve, as expressed in Equation 10. This definition is based on the observation that the initial stiffness is best represented by the secant modulus at a low fraction of the peak load, which avoids the nonlinear effects that occur near the peak.

$$K_e = \frac{0.4F_{\max}}{D_{0.4}} \quad (10)$$

In this equation, $D_{0.4}$ represents the displacement at which the force reaches 40% of the peak force on the ascending branch. This point is carefully selected because it lies in the approximately linear region of the response, ensuring that the computed stiffness is representative of the undamaged behaviour.

2.5.2 Ultimate Point

The ultimate point (D_u, F_u) is a critical parameter that defines the deformation capacity of the structure. According to the ASTM E2126 standard, the ultimate point is defined as the first point after the peak where the force drops below 80% of the peak force, as shown in Equation 11. This 80% criterion is based on the concept that the structure has lost 20% of its peak strength, which is considered the practical limit of its load-carrying capacity.

$$F_u = 0.8F_{\max} \quad (11)$$

The corresponding displacement D_u is obtained by linear interpolation between the two points bracketing the 80% force level, as expressed in Equation 12. This interpolation procedure ensures accurate determination of the ultimate displacement even when the data points do not exactly coincide with the 80% force level.

$$D_u = D_i + \frac{F_u - F_i}{F_{i+1} - F_i}(D_{i+1} - D_i) \quad (12)$$

In cases where the data does not exhibit a post-peak drop of 20%, the ultimate point is conservatively taken as the last available data point. This fallback assumption, expressed in Equation 13, ensures that the analysis can proceed even when the data is incomplete.

$$(D_u, F_u) = (x_{\text{last}}, y_{\text{last}}) \quad (13)$$

2.5.3 Yield Point

The yield point (D_y, F_y) is determined by solving the equal energy equation, which is the cornerstone of the EEEP method. This equation, presented in Equation 14, expresses the fundamental principle that the area under the idealized bilinear curve must equal the area under the actual curve up to the ultimate point.

$$\frac{1}{2}K_e D_y^2 + F_y(D_u - D_y) = \int_0^{D_u} F_{\text{actual}}(D) dD \quad (14)$$

The left-hand side of this equation represents the area under the idealized elastic-perfectly-plastic curve, which consists of a linear elastic portion up to yielding followed by a perfectly plastic plateau. The right-hand side represents the area under the actual backbone curve, which is computed numerically using the trapezoidal rule.

Solving this equation for the yield force yields Equation 15, which provides a closed-form solution for the yield force in terms of the initial stiffness, ultimate displacement, and total energy.

$$F_y = K_e D_u - \sqrt{(K_e D_u)^2 - 2K_e E} \quad (15)$$

In this equation, E represents the total area under the actual curve up to the ultimate displacement, as defined in Equation 16. This area is computed by numerical integration of the backbone curve.

$$E = \int_0^{D_u} F_{\text{actual}}(D) dD \quad (16)$$

The yield displacement is then obtained from the yield force using the initial stiffness, as shown in Equation 17. This relationship follows directly from the definition of the initial stiffness as the slope of the elastic portion of the idealized curve.

$$D_y = \frac{F_y}{K_e} \quad (17)$$

2.5.4 Ductility

The displacement ductility, defined in Equation 18, is a fundamental measure of the deformation capacity of the structure. It represents the ratio of the ultimate displacement to the yield displacement and provides a quantitative measure of the structure's ability to deform beyond yielding without collapse.

$$\mu = \frac{D_u}{D_y} \quad (18)$$

This parameter is of paramount importance in seismic design because it indicates how much the structure can deform inelastically while still maintaining its load-carrying capacity. Higher ductility values generally indicate better seismic performance.

2.6 Energy Dissipation

The energy dissipated in a single hysteresis loop is a critical measure of the structure's energy absorption capacity. This energy is computed using the shoelace formula, which gives the exact area enclosed by the loop when the data points are connected by straight lines. The formula is presented in Equation 19.

$$E_j = \frac{1}{2} \left| \sum_{k=1}^{N_j} (D_k F_{k+1} - D_{k+1} F_k) \right| \quad (19)$$

In this equation, the summation is performed over all points in the cycle, with the loop being closed so that $F_{N_j+1} = F_1$ and $D_{N_j+1} = D_1$. This ensures that the path is closed and the area is properly computed. The absolute value is taken to ensure that the area is positive regardless of the direction of integration.

The cumulative energy dissipation, defined in Equation 20, represents the total energy dissipated by the structure up to cycle j . This cumulative measure provides insight into the total energy absorption capacity of the structure and is important for understanding the overall seismic performance.

$$E_{\text{cum},j} = \sum_{k=1}^j E_k \quad (20)$$

2.7 Stiffness Degradation

The secant stiffness for cycle j is a measure of the effective stiffness of the structure during that cycle. It is defined as the slope of the line connecting the positive and negative peak points of the loop, as shown in Equation 21. This definition provides a consistent measure of stiffness that can be compared across cycles.

$$K_j = \frac{F_j^{(+)} - F_j^{(-)}}{D_j^{(+)} - D_j^{(-)}} \quad (21)$$

The stiffness retention, defined in Equation 22, expresses the current stiffness as a percentage of the initial stiffness. This parameter provides a clear indication of the degree of stiffness degradation that has occurred up to cycle j .

$$K_{\text{ret},j} = \frac{K_j}{K_1} \times 100\% \quad (22)$$

The progressive reduction in stiffness with increasing cycles is a manifestation of cumulative damage in the structure. This degradation is typically more pronounced in the early cycles and tends to stabilize as the structure reaches a steady-state damage condition.

3 Software Architecture and Design

3.1 System Architecture

The application follows a modular architecture with clear separation of concerns, which enhances maintainability, testability, and extensibility. The architecture consists of four distinct layers, each with specific responsibilities as described in Table 4.

Table 4: System Architecture Layers

Layer	Responsibility
Data Layer	Handles all file I/O operations, data parsing from TXT files, and in-memory data storage
Processing Layer	Implements all mathematical computations and algorithms, including backbone extraction and EEEP idealization
Presentation Layer	Manages the GUI, interactive plotting, and all user interaction events
Persistence Layer	Handles settings management, output generation, and CSV file writing

This layered architecture ensures that each component has a well-defined purpose and can be developed, tested, and maintained independently. The separation of concerns also facilitates future enhancements and modifications to individual layers without affecting the others.

3.2 Class Structure

The main application class `BackboneGUI` is the central component that manages all GUI elements and coordinates the processing workflow. This class is designed with a clear structure that organizes the various components of the application.

3.2.1 Data Management

The data management attributes of the main class are responsible for storing and organizing all data processed by the application. The code snippet below shows the key data structures used in the application:

```
class BackboneGUI:
    def __init__(self):
        self.results = {} # stem -> plot_data dict
        self.summary_rows = [] # list of dicts
        self.summary_df = pd.DataFrame()
        self.log_queue = queue.Queue()
        self.worker_thread = None
        self.cancel_event = threading.Event()
```

The **results** dictionary stores all plot data for each processed specimen, with the specimen name as the key and the plot data as the value. The **summary_rows** list contains dictionaries representing each row of the summary table. The **summary_df** is a Pandas DataFrame that provides convenient data manipulation capabilities. The **log_queue** is a thread-safe queue used for communication between the worker thread and the main GUI thread.

3.2.2 Processing Pipeline

The processing pipeline consists of eight distinct stages that transform the raw input data into the final results. Each stage has a specific purpose and contributes to the overall analysis process. Table 5 provides a detailed description of each stage.

Table 5: Processing Pipeline Stages

Stage	Description
File Discovery	Scans the input folder for TXT files and validates their existence and readability
Data Loading	Parses each TXT file, extracts displacement and force arrays, and performs data validation
Backbone Extraction	Identifies reversal points, extracts the monotonic envelope, and constructs the backbone curve
Branch Separation	Separates the backbone into positive and negative branches for independent analysis
Branch Averaging	Mirrors the negative branch and computes the average branch for bilinear idealization
Property Computation	Calculates all engineering properties including stiffness, yield point, and ductility
Bilinear Idealization	Applies the EEEP method to determine the idealized elastic-perfectly-plastic curve
Cycle Analysis	Identifies individual cycles and computes cycle-specific metrics
Output Generation	Creates plots, CSV files, and updates the summary table

3.3 Dependency Management

The application relies on several carefully selected Python packages that provide essential functionality. These dependencies are listed in Table 6, along with their specific purposes in the application.

Table 6: Software Dependencies

Package	Purpose
NumPy	Provides efficient numerical operations, array manipulation, and linear algebra capabilities
Pandas	Offers advanced data management, CSV I/O operations, and sophisticated data structures
Matplotlib	Enables high-quality plotting, visualization, and figure generation with publication-ready output
Tkinter	Serves as the GUI framework, providing all windowing and widget functionality

3.4 Data Flow Diagram

The data flow through the application follows a well-defined sequence that ensures proper handling of data at each stage. The process begins with the user selecting input and output folders. The application then scans the input folder for TXT files and processes each file through the complete pipeline. The key steps in this data flow are:

1. The user selects input and output folders through the GUI
2. The application scans the input folder for all TXT files
3. For each file, the following processing steps are performed:
 - (a) Load displacement and force data from the TXT file
 - (b) Extract reversal points to identify turning points in the hysteresis curve
 - (c) Build the monotonic envelope using the clustering algorithm
 - (d) Extract the full backbone curve by combining positive and negative branches
 - (e) Separate the backbone into positive and negative branches
 - (f) Mirror the negative branch to the positive quadrant
 - (g) Compute the average branch by interpolation and averaging
 - (h) Calculate branch properties including peak, ultimate, and yield points
 - (i) Generate the EEEP bilinear idealization
 - (j) Identify individual cycles and compute cycle-specific metrics
 - (k) Generate plots and CSV outputs for the specimen
 - (l) Store results in memory for interactive visualization
4. Update the summary table with all results
5. Enable interactive visualization of results

3.5 Threading Model

The application employs a sophisticated multi-threaded architecture that maintains GUI responsiveness during intensive computational operations. This threading model is essential for providing a smooth user experience, especially when processing large datasets. Table 7 describes the components of this threading model.

The threading model ensures that the GUI remains responsive and interactive even during long processing operations. The worker thread runs in the background, periodically updating the progress bar and log display through the thread-safe log queue. The cancel event mechanism provides a robust way for users to abort processing if needed.

Table 7: Threading Model Components

Component	Description
Main Thread	Handles all GUI events, updates the interface, and maintains responsiveness
Worker Thread	Performs the intensive processing operations in the background without blocking the GUI
Log Queue	Provides thread-safe communication between the worker thread and the main thread for log messages
Cancel Event	Allows the user to gracefully cancel processing operations at any time

4 Mathematical Algorithms and Implementations

This chapter provides a comprehensive and detailed description of all mathematical algorithms implemented in the code, with complete explanations of the theoretical foundations and practical implementation details.

4.1 Data Loading and Validation

The data loading function `load_hysteresis_data(txt_path)` implements a robust algorithm for reading and validating input data. This algorithm follows a well-defined sequence of steps to ensure data integrity before processing. Table 8 describes each step of this algorithm.

Table 8: Data Loading Algorithm Steps

Step	Description
File Reading	Opens the TXT file and reads all lines into memory
Header Skipping	Skips the first row which contains column headers
Column Splitting	Splits each line by tab or whitespace delimiter into displacement and force columns
Data Conversion	Converts string values to floating-point numbers
Validation	Checks for NaN values and ensures at least 3 valid data points

4.1.1 File Parsing

The file parsing process involves reading the TXT file and converting its contents into numerical arrays. The parsing algorithm first attempts to read the file using tab as the delimiter, and if that fails, it falls back to using any whitespace as the delimiter. This flexibility ensures that the application can handle various file formats commonly encountered in experimental data. The parsing process is mathematically described in Equation 23, which shows the sequence of operations performed to convert the raw text file into numerical arrays.

$$\begin{aligned}
 &\text{Read file} \rightarrow \text{Skip header row} \\
 &\quad \rightarrow \text{Split into columns} \\
 &\quad \rightarrow \text{Convert to numeric arrays} \\
 &\quad \rightarrow \text{Validate data integrity}
 \end{aligned} \tag{23}$$

4.1.2 Data Validation

The data validation step is critical for ensuring the quality and reliability of the analysis results. The validation criteria, expressed in Equation 24, require that the data contain at least three valid points with no missing or infinite values.

$$\text{Valid if } N \geq 3 \text{ and no NaN values and finite values} \quad (24)$$

4.2 Reversal Point Detection

The function `find_reversal_points(displacement, force)` implements a sophisticated algorithm for identifying turning points in the hysteresis curve. These turning points, also known as reversal points, are essential for extracting the backbone curve and identifying individual cycles. The algorithm follows a systematic process that is described in Table 9.

Table 9: Reversal Point Detection Algorithm

Step	Description
Gradient Calculation	Computes the difference between consecutive displacement values to determine the direction of motion
Sign Determination	Determines the sign of each gradient to identify the direction of movement (positive or negative)
Zero Handling	Handles zero gradients by forward filling from the nearest non-zero gradient
Turning Point Detection	Identifies turning points where the sign of the gradient changes
Endpoint Handling	Marks the first and last points as turning points to ensure complete cycle identification

4.2.1 Gradient Calculation

The first step in reversal point detection is the computation of the displacement gradient. This gradient represents the change in displacement between consecutive data points and is fundamental to identifying the direction of motion. The gradient is calculated using Equation 25, which computes the difference between each pair of consecutive displacements.

$$\Delta D_i = D_{i+1} - D_i, \quad i = 1, \dots, N - 1 \quad (25)$$

4.2.2 Sign Determination

Once the gradients have been computed, the next step is to determine the sign of each gradient. The sign function, defined in Equation 26, returns 1 for positive gradients (increasing displacement), -1 for negative gradients (decreasing displacement), and 0 for zero gradients (no change in displacement).

$$S_i = \text{sign}(\Delta D_i) = \begin{cases} 1, & \Delta D_i > 0 \\ 0, & \Delta D_i = 0 \\ -1, & \Delta D_i < 0 \end{cases} \quad (26)$$

Zero gradients, which can occur due to data acquisition issues or numerical rounding, are handled by forward filling from the nearest non-zero gradient, as shown in Equation 27. This

ensures that the direction of motion is consistently defined even in the presence of zero-gradient points.

$$S_i^{\text{filled}} = \text{fill}(S_i) \quad (27)$$

4.2.3 Turning Point Detection

A turning point is identified when the sign of the gradient changes, indicating that the direction of motion has reversed. This detection is expressed mathematically in Equation 28, which assigns a value of 1 to points where the sign changes and 0 otherwise.

$$T_i = \begin{cases} 1, & S_i \neq S_{i-1} \\ 0, & \text{otherwise} \end{cases} \quad (28)$$

The first and last points of the dataset are always considered turning points, as expressed in Equation 29. This ensures that the algorithm identifies complete cycles, even when the data does not start and end at the same displacement level.

$$T_0 = 1, \quad T_{N-1} = 1 \quad (29)$$

4.3 Monotonic Envelope Extraction

The function `monotonic_envelope(x, y, cluster_tol=0.02)` implements a robust algorithm for extracting the monotonic envelope from the data. This algorithm is designed to identify the outermost points of the response envelope while ensuring monotonicity. Table 10 describes the steps of this algorithm.

Table 10: Monotonic Envelope Extraction Algorithm

Step	Description
Sorting	Sorts all data points by absolute displacement, ensuring the envelope is built from the smallest to the largest displacement
Clustering	Groups points that are within the specified tolerance, preventing noise from affecting the envelope
Reduction	For each cluster, retains only the point with the maximum absolute force, representing the true envelope
Monotonicity	Retains only those points where the absolute displacement strictly increases, ensuring a well-defined function

4.3.1 Sorting by Absolute Displacement

The first step in the monotonic envelope extraction is sorting the data by absolute displacement. This sorting operation, expressed in Equation 30, ensures that the envelope is built from the smallest to the largest displacements, which is essential for monotonicity.

$$\text{idx} = \text{argsort}(|x|) \quad (30)$$

4.3.2 Clustering

The clustering operation groups data points that are within a specified tolerance of each other. This tolerance, defined in Equation 31, is set to 2% of the maximum absolute displacement.

This approach prevents noise and minor variations in the data from causing multiple closely spaced points to appear in the envelope.

$$\tau = \text{cluster_tol} \times \max(|x|) \quad (31)$$

The clustering algorithm itself is described in Equation 32, which shows how points are progressively added to clusters until the tolerance is exceeded, at which point a new cluster is started.

$$\begin{aligned} &\text{Initialize cluster } C_1 = \{(x_1, |y_1|)\} \\ &\text{For } i = 2, \dots, N : \\ &\quad \text{If } x_i - x_{\text{last}} \leq \tau : \text{ Add to current cluster} \\ &\quad \text{Else: Start new cluster} \end{aligned} \quad (32)$$

4.3.3 Cluster Reduction

For each cluster of points, the reduction step retains only the point with the maximum absolute force, as shown in Equation 33. This ensures that the envelope represents the true maximum forces achieved in each displacement region.

$$(\bar{x}_j, \bar{y}_j) = \arg \max_{i \in C_j} |y_i| \quad (33)$$

4.3.4 Monotonicity Enforcement

The final step in the envelope extraction is the enforcement of monotonicity. Only those points where the absolute displacement strictly increases are retained, as shown in Equation 34. This ensures that the envelope is a well-defined function, which is necessary for subsequent analysis.

$$\text{Keep } (\bar{x}_j, \bar{y}_j) \text{ if } \bar{x}_j > \max_{k < j} \bar{x}_k \quad (34)$$

4.4 Backbone Curve Construction

The function `build_backbone(displacement, force)` constructs the complete backbone curve by combining the positive and negative branches of the response. This construction follows a systematic process that is described in Table 11.

Table 11: Backbone Curve Construction Algorithm

Step	Description
Branch Separation	Separates the data points into positive and negative branches based on the sign of the displacement
Envelope Extraction	Extracts the monotonic envelope for each branch independently
Combination	Combines the two branches with the origin point to form the complete backbone curve

4.4.1 Branch Separation

The branch separation process divides the data into positive and negative branches based on the sign of the displacement. This separation, expressed in Equation 36, creates two independent datasets that can be analyzed separately.

$$P = \{(x_i, y_i) \mid x_i > 0\} \quad (35)$$

$$N = \{(x_i, y_i) \mid x_i < 0\} \quad (36)$$

4.4.2 Monotonic Envelope Extraction

The monotonic envelope is extracted independently for each branch using the algorithm described in the previous section. For the negative branch, the absolute displacement is used to ensure consistent processing, as shown in Equation 38.

$$(P_x, P_y) = \text{monotonic_envelope}(P_x, P_y) \quad (37)$$

$$(N_x, N_y) = \text{monotonic_envelope}(-N_x, -N_y) \quad (38)$$

4.4.3 Combined Backbone

The complete backbone curve is constructed by combining the negative branch, the origin point, and the positive branch, as shown in Equation 39. This combination creates a continuous curve that represents the full response envelope.

$$B = (N_x, N_y) \cup (0, 0) \cup (P_x, P_y) \quad (39)$$

4.5 Mirroring Negative Branch

The function `mirror_negative_branch(neg_x, neg_y)` implements the mirroring operation that reflects the negative branch onto the positive quadrant. This mirroring is essential for computing the average branch and for comparing the positive and negative responses.

4.5.1 Reversal and Sign Change

The mirroring operation involves reversing the order of the negative branch and negating both the displacement and force values, as shown in Equation 41. This transformation results in a curve in the positive quadrant that represents the mirrored negative branch.

$$M_x = -\text{reverse}(N_x) \quad (40)$$

$$M_y = -\text{reverse}(N_y) \quad (41)$$

The resulting curve $M = (M_x, M_y)$ lies entirely in the positive quadrant, with displacement and force values that are the absolute values of the corresponding negative branch points. This mirroring allows for direct comparison with the positive branch.

4.6 Average Branch Calculation

The function `average_branch(pos_x, pos_y, mirror_x, mirror_y)` computes the average branch by interpolating both the positive and mirrored negative branches onto a common grid and averaging the results. Table 12 describes the steps of this algorithm.

4.6.1 Common Grid

The common grid is created by taking the union of the displacement values from both branches, as shown in Equation 42. This ensures that the average branch is computed at all points where either branch has data.

$$G = \text{unique}(P_x \cup M_x) \quad (42)$$

Table 12: Average Branch Calculation Algorithm

Step	Description
Grid Creation	Creates a common grid from the union of both branch displacement values
Interpolation	Interpolates both branches onto the common grid using linear interpolation
Averaging	Computes the average of the two interpolated curves at each grid point

4.6.2 Interpolation

Both branches are interpolated onto the common grid using linear interpolation, as shown in Equation 44. This interpolation provides values at all grid points, even where a branch does not originally have data.

$$P_y^{(G)} = \text{interp}(G, P_x, P_y) \quad (43)$$

$$M_y^{(G)} = \text{interp}(G, M_x, M_y) \quad (44)$$

4.6.3 Averaging

The average branch is computed by taking the arithmetic mean of the two interpolated curves, as shown in Equation 45. This average represents the typical behaviour of the structure in both loading directions.

$$A_y^{(G)} = \frac{P_y^{(G)} + M_y^{(G)}}{2} \quad (45)$$

4.7 Branch Property Computation

The function `compute_branch_properties(x, y)` computes all engineering properties for a given branch. This comprehensive computation follows the ASTM E2126 EEEP method and provides all parameters necessary for characterizing the cyclic behaviour. Table 13 summarizes the computed properties.

Table 13: Branch Property Computation

Property	Description
Peak Point	Maximum force and corresponding displacement, representing the peak strength
Ultimate Point	Point at 80% of peak force after the peak, representing the deformation capacity
Initial Stiffness	Secant stiffness through 40% of peak force, representing the elastic behaviour
Energy	Area under the curve up to the ultimate point, representing the energy absorption
Bilinear Idealization	EEEEP yield point and ductility, providing the idealized response

4.7.1 Peak Point

The peak point is found by identifying the maximum force and its corresponding displacement, as shown in Equation 46. This point represents the peak strength of the structure.

$$i_{\max} = \arg \max y, \quad F_{\max} = y_{i_{\max}}, \quad D_{\max} = x_{i_{\max}} \quad (46)$$

4.7.2 Ultimate Point

The ultimate point is found by searching for the first post-peak point where the force drops below a specified fraction of the peak force. The default fraction is 80%, which is consistent with the ASTM E2126 standard, as shown in Equation 47.

$$F_u = \text{degrade_frac} \times F_{\max} \quad (47)$$

The displacement at the ultimate point is obtained by linear interpolation, as shown in Equation 48. This interpolation ensures accurate determination of the ultimate displacement even when the data points do not exactly coincide with the 80% force level.

$$D_u = x_i + \frac{F_u - y_i}{y_{i+1} - y_i} (x_{i+1} - x_i) \quad (48)$$

4.7.3 Initial Stiffness

The initial stiffness is found by locating the point on the ascending branch where the force reaches 40% of the peak force, as shown in Equation 49. This point lies in the approximately linear region of the response.

$$F_{0.4} = 0.4 F_{\max} \quad (49)$$

The displacement at this force is obtained by linear interpolation, as shown in Equation 50. The initial stiffness is then computed as the ratio of the force to the displacement, as shown in Equation 51.

$$D_{0.4} = x_i + \frac{F_{0.4} - y_i}{y_{i+1} - y_i} (x_{i+1} - x_i) \quad (50)$$

$$K_e = \frac{F_{0.4}}{D_{0.4}} \quad (51)$$

4.7.4 Energy Dissipation

The energy dissipated up to the ultimate point is computed using the trapezoidal rule, as shown in Equation 52. This numerical integration provides an accurate estimate of the area under the curve.

$$E = \int_0^{D_u} y(x) dx \approx \sum_{i=1}^{n-1} \frac{y_i + y_{i+1}}{2} \Delta x_i \quad (52)$$

4.7.5 EEEP Bilinear Idealization

The EEEP bilinear idealization is computed using the equal energy principle. The yield force is given by Equation 53, which provides a closed-form solution in terms of the initial stiffness, ultimate displacement, and total energy.

$$F_y = K_e D_u - \sqrt{(K_e D_u)^2 - 2K_e E} \quad (53)$$

The yield displacement is then obtained from the yield force using the initial stiffness, as shown in Equation 54. This relationship follows directly from the definition of the initial stiffness.

$$D_y = \frac{F_y}{K_e} \quad (54)$$

The displacement ductility is computed as the ratio of the ultimate displacement to the yield displacement, as shown in Equation 55. This ductility is a key measure of the deformation capacity of the structure.

$$\mu = \frac{D_u}{D_y} \quad (55)$$

4.8 Cycle Identification

The function `identify_cycles(dispatch, force)` implements a robust algorithm for identifying individual loading cycles in the time-ordered data. This algorithm follows a systematic process that is described in Table 14.

Table 14: Cycle Identification Algorithm

Step	Description
Peak Detection	Identifies positive displacement peaks from the turning points
Segmentation	Segments the data between consecutive positive peaks to form individual cycles
Peak Extraction	Extracts the positive and negative force peaks within each cycle

4.8.1 Positive Peak Detection

Positive displacement peaks are identified from the turning points, as shown in Equation 56. Only those turning points with positive displacement are considered, ensuring that cycles are properly defined.

$$P_{\text{peak}} = \{i \mid T_i = 1 \text{ and } D_i > 0\} \quad (56)$$

4.8.2 Cycle Segmentation

Each cycle is defined as the segment of data between consecutive positive displacement peaks, as shown in Equation 57. This definition ensures that each cycle contains exactly one positive and one negative excursion.

$$\text{Cycle}_j = \{(D_i, F_i) \mid i_{\text{peak},j} \leq i \leq i_{\text{peak},j+1}\} \quad (57)$$

4.8.3 Peak Extraction

Within each cycle, the positive and negative force peaks are identified, as shown in Equation 59. These peaks are used for computing the secant stiffness and other cycle-specific metrics.

$$i_{\text{pos}} = \arg \max F_i \quad (58)$$

$$i_{\text{neg}} = \arg \min F_i \quad (59)$$

4.9 Cycle Metrics Computation

The function `compute_cycle_metrics(cycles)` computes a comprehensive set of metrics for each identified cycle. These metrics provide detailed insight into the cyclic behaviour and degradation. Table 15 summarizes the computed metrics.

Table 15: Cycle Metrics Formulas

Metric	Formula
Secant Stiffness	$K_j = \frac{F_j^{(+)} - F_j^{(-)}}{D_j^{(+)} - D_j^{(-)}}$
Stiffness Retention	$K_{\text{ret},j} = \frac{K_j}{K_1} \times 100\%$
Dissipated Energy	$E_j = \frac{1}{2} \left \sum_{k=1}^{N_j} (D_k F_{k+1} - D_{k+1} F_k) \right $
Cumulative Energy	$E_{\text{cum},j} = \sum_{k=1}^j E_k$

4.9.1 Secant Stiffness

The secant stiffness for cycle j is computed as the slope of the line connecting the positive and negative peak points, as shown in Equation 60. This stiffness represents the effective stiffness of the structure during that cycle.

$$K_j = \frac{F_j^{(+)} - F_j^{(-)}}{D_j^{(+)} - D_j^{(-)}} \quad (60)$$

4.9.2 Stiffness Retention

The stiffness retention, expressed as a percentage of the initial stiffness, is computed as shown in Equation 61. This metric provides a clear indication of the stiffness degradation that has occurred.

$$K_{\text{ret},j} = \frac{K_j}{K_1} \times 100\% \quad (61)$$

4.9.3 Dissipated Energy

The energy dissipated in each cycle is computed using the shoelace formula, as shown in Equation 62. This formula gives the exact area enclosed by the cycle when the data points are connected by straight lines.

$$E_j = \frac{1}{2} \left| \sum_{k=1}^{N_j} (D_k F_{k+1} - D_{k+1} F_k) \right| \quad (62)$$

4.9.4 Cumulative Energy

The cumulative energy dissipation is computed by summing the energy dissipated in each cycle, as shown in Equation 63. This cumulative energy represents the total energy absorbed by the structure.

$$E_{\text{cum},j} = \sum_{k=1}^j E_k \quad (63)$$

5 GUI Implementation and Features

5.1 Layout Structure

The GUI is organized into a hierarchical layout structure that provides a clear and intuitive user experience. The top-level structure is shown in the code listing below:

```
BackboneGUI (Tk)
+-- Header Frame (branding)
+-- Folder Selection Panel
+-- Notebook (Tabs)
    +-- Plot Viewer Tab
        +-- Navigation Bar
        +-- View Controls
        +-- Export Controls
        +-- Canvas Frame (plot)
        +-- Engineering Information Panel
    +-- Processing Log Tab
        +-- Log Controls
        +-- Log Text Area
    +-- Summary Table Tab
        +-- Search and Filter Controls
        +-- Treeview Table
+-- Status Bar
+-- Footer
```

This hierarchical organization ensures that related controls are grouped together, making the interface easy to navigate and use.

5.2 Interactive Plot Features

The plot viewer provides a rich set of interactive features that facilitate detailed data exploration and analysis. These features are summarized in Table 16.

5.2.1 Mouse Interaction

The mouse interaction is implemented through event handlers that respond to different mouse actions. Table 17 describes the response to each type of mouse event.

5.2.2 Crosshair Cursor

The crosshair cursor consists of two perpendicular lines that intersect at the mouse position, as shown in Equation 64. This feature aids in precise data reading and alignment.

$$\begin{aligned} L_{\text{horizontal}} &: y = y_{\text{mouse}} \\ L_{\text{vertical}} &: x = x_{\text{mouse}} \end{aligned} \quad (64)$$

Table 16: Interactive Plot Features

Feature	Description
Hover	Displays the current cursor coordinates in the status bar for precise data reading
Click	Highlights the nearest data point and displays its coordinates in the information panel
Pan	Allows click-and-drag translation of the plot view for exploring different regions
Zoom	Enables scroll wheel zoom for detailed examination of specific regions
Crosshair	Provides dynamic horizontal and vertical guide lines that follow the mouse cursor

Table 17: Mouse Interaction Response

Event	Action
Motion	Updates the status bar with the current cursor coordinates
Button Press	Finds and highlights the nearest data point to the click location
Button Drag	Pans the plot view in the direction of the drag
Scroll	Zooms in or out around the cursor position

5.2.3 Data Inspection

Clicking on the plot triggers a nearest-point search algorithm, as shown in Equation 65. This algorithm finds the data point with the minimum Euclidean distance to the click location.

$$\text{idx} = \arg \min_i \|(x_i, y_i) - (x_{\text{mouse}}, y_{\text{mouse}})\|_2 \quad (65)$$

5.3 Engineering Information Panel

The engineering information panel displays a comprehensive set of parameters for the selected specimen. These parameters are listed in Table 18, along with their descriptions.

Table 18: Information Panel Fields

Field	Description
Specimen	The name of the specimen being displayed
Peak Force	Maximum force F_{max} attained during the test, in kilonewtons (kN)
Peak Displacement	Displacement at which the peak force was attained, in millimetres (mm)
Yield Force	Yield force F_y determined by the EEEP method, in kilonewtons (kN)
Yield Displacement	Yield displacement D_y determined by the EEEP method, in millimetres (mm)
Ultimate Force	Ultimate force F_u at 80% of peak force, in kilonewtons (kN)

Field	Description
Ultimate Displacement	Ultimate displacement D_u at 80% of peak force, in millimetres (mm)
Initial Stiffness	Initial stiffness K_e through 40% of peak force, in kilonewtons per millimetre (kN/mm)
Energy Dissipation	Total energy E dissipated up to the ultimate point, in kilonewton-millimetres (kNmm)
Ductility	Ductility ratio $\mu = D_u/D_y$
Number of Cycles	Total number of identified cycles
Processing Status	Indicates whether processing succeeded (OK) or failed (FAIL), with notes

5.4 Processing Log Implementation

The processing log provides real-time feedback during batch processing using a color-coding system that makes it easy to identify the status of each operation. The color coding is described in Table 19.

Table 19: Processing Log Color Coding

Color	Meaning
Green (OK)	Indicates that a file was processed successfully without errors
Red (FAIL)	Indicates that a file processing error occurred
Blue (Header)	Indicates section headers and summary information
Gray (Dim)	Indicates informational or verbose messages

5.5 Summary Table Implementation

The summary table uses a Treeview widget that provides comprehensive data presentation and interaction capabilities. Table 20 describes the main features of this table.

Table 20: Summary Table Features

Feature	Description
Sorting	Click on column headers to sort the table in ascending or descending order
Filtering	Use the search box to filter rows by any column content
Highlighting	Automatic highlighting of maximum and minimum values for selected numeric columns
Selection	Multiple row selection for bulk operations
Export	Export selected rows to CSV format for further analysis

5.5.1 Sorting

The sorting feature is implemented using Pandas, as shown in Equation 66. Clicking on a column header sorts the table by that column, with alternating ascending and descending order.

$$\text{df_sorted} = \text{df.sort_values}(by = \text{col}, \text{ascending} = \text{asc}) \quad (66)$$

5.5.2 Filtering

The filtering feature uses string matching to show only rows that contain the search query, as shown in Equation 67. This allows users to quickly find specific specimens or data.

$$\text{mask} = \text{df.apply}(\text{row} \rightarrow \text{query} \in \text{row.astype(str)}) \quad (67)$$

5.5.3 Highlighting

The highlighting feature automatically identifies and highlights the maximum and minimum values in numeric columns, as shown in Equation 68. This makes it easy to identify extreme values.

$$\begin{aligned} i_{\max} &= \arg \max \text{series} \\ i_{\min} &= \arg \min \text{series} \end{aligned} \quad (68)$$

6 Output Generation and File Formats

6.1 Directory Structure

The application automatically generates a comprehensive directory structure for organizing all output files. This structure, shown in the code listing below, ensures that different types of outputs are properly categorized and easy to locate.

```
[Output Folder]/
+-- All Backbone Curves-Plots/ # Backbone curve plots in PNG format
+-- All Backbone Curves-CSVs/ # Backbone curve data in CSV format
+-- Positive-Negative-Average Plots/ # Branch comparison plots in PNG format
+-- Positive-Negative-Average CSVs/ # Branch data in CSV format
+-- Bilinear Idealization Plots/ # Bilinear idealization plots in PNG format
+-- Bilinear Idealization CSVs/ # Bilinear data in CSV format
+-- Stiffness Degradation Plots/ # Stiffness degradation plots in PNG format
+-- Stiffness Degradation CSVs/ # Cyclic properties data in CSV format
+-- Energy Dissipation Plots/ # Energy dissipation plots in PNG format
+-- Energy Dissipation CSVs/ # Energy dissipation data in CSV format
+-- Summary/ # Master summary CSV file
```

6.2 CSV File Formats

The application generates several CSV files with standardized formats. Table 21 describes the columns in each CSV file type.

Table 21: CSV File Formats

File Type	Columns
Backbone Curve	displacement_mm, force_kN
PNA	displacement_mm, positive_force_kN, negative_force_mirrored_kN, average_force_kN
Bilinear Idealization	displacement_mm, force_kN
Cyclic Properties	cycle_number, stiffness_kN_per_mm, stiffness_pct_of_initial, energy_dissipated_kN_mm, cumulative_energy_kN_mm

6.2.1 Backbone Curve CSV

The backbone curve CSV file contains the displacement-force coordinates of the extracted backbone curve, as shown in Equation 69. This file provides the complete envelope data for further analysis.

$$\text{Columns} = [\text{displacement_mm}, \text{force_kN}] \quad (69)$$

6.2.2 Positive/Negative/Average CSV

The PNA CSV file contains the positive, mirrored negative, and average branch data, as shown in Equation 70. This file provides detailed information about the symmetric and average behaviour.

$$\text{Columns} = [\text{displacement_mm}, \text{positive_force_kN}, \text{negative_force_mirrored_kN}, \text{average_force_kN}] \quad (70)$$

6.2.3 Bilinear Idealization CSV

The bilinear idealization CSV file contains the points defining the EEEP bilinear idealization, as shown in Equation 71. This file provides the idealized response curve.

$$\text{Columns} = [\text{displacement_mm}, \text{force_kN}] \quad (71)$$

6.2.4 Cyclic Properties CSV

The cyclic properties CSV file contains all cycle-specific metrics, as shown in Equation 72. This file provides detailed information about stiffness degradation and energy dissipation.

$$\text{Columns} = [\text{cycle_number}, \text{stiffness_kN_per_mm}, \text{stiffness_pct_of_initial}, \text{energy_dissipated_kN_mm}, \text{cumulative_energy_dissipated_kN_mm}] \quad (72)$$

6.3 Plot Generation

The application generates several types of plots with specific styling to ensure clarity and professionalism. Table 22 describes the styling of each plot type.

Table 22: Plot Types and Styling

Plot Type	Styling Description
Backbone Curve	Raw data in light gray with 0.8 linewidth; Backbone in crimson red with 2.2 linewidth and markers
Bilinear Idealization	Average backbone in black with 2.0 linewidth; Bilinear in seagreen with dashed line and markers
Stiffness Degradation	Bar chart in blue with black edges showing per-cycle stiffness
Energy Dissipation	Bar chart in orange with black edges showing per-cycle energy
Cumulative Energy	Bar chart in green with black edges showing cumulative energy

6.3.1 Backbone Curve Plot

The backbone curve plot displays both the raw hysteresis data and the extracted backbone curve. The raw data is shown in light gray to provide context, while the backbone curve is highlighted in crimson red with markers. A grid with 30% opacity is included for reference, and the origin is marked with a black dot for clarity.

6.3.2 Bilinear Idealization Plot

The bilinear idealization plot overlays the EEEP idealization on the average backbone curve. The average backbone is shown in black, while the bilinear idealization is shown in seagreen with a dashed line and markers. The peak and ultimate points are marked with red and purple dots, respectively, for easy identification.

6.3.3 Stiffness Degradation Plot

The stiffness degradation plot is a bar chart that shows the secant stiffness for each cycle. The bars are shown in blue with black edges for clarity. This plot provides a clear visual representation of stiffness degradation with increasing cycles.

$$\text{Bars: } (j, K_j) \text{ for } j = 1, \dots, n \quad (73)$$

6.3.4 Energy Dissipation Plot

The energy dissipation plot is a bar chart that shows the energy dissipated in each cycle. The bars are shown in orange with black edges. This plot provides insight into the energy absorption capacity of the structure.

$$\text{Bars: } (j, E_j) \text{ for } j = 1, \dots, n \quad (74)$$

6.4 Master Summary CSV

The master summary CSV file provides a comprehensive overview of all processed specimens. Table 23 lists all columns in this file, organized by category.

Table 23: Summary CSV Columns

Column	Description
Specimen	Name of the specimen derived from the input filename
Pos_Fmax_kN	Positive branch peak force in kilonewtons
Pos_Dmax_mm	Positive branch peak displacement in millimetres
Pos_Dy_mm	Positive branch yield displacement in millimetres
Pos_Fy_kN	Positive branch yield force in kilonewtons
Pos_Du_mm	Positive branch ultimate displacement in millimetres
Pos_Fu_kN	Positive branch ultimate force in kilonewtons
Pos_Ke_kN_per_mm	Positive branch initial stiffness in kilonewtons per millimetre
Pos_Energy_kN_mm	Positive branch energy dissipation in kilonewton-millimetres
Pos_Ductility	Positive branch ductility ratio
Pos_Notes	Positive branch notes and warnings
Neg_*	All negative branch (mirrored) properties with the same naming convention
Avg_*	All average branch properties with the same naming convention

Column	Description
Num.Cycles	Total number of identified cycles
Total_Energy_Dissipated_kNm	Total cumulative energy dissipation in kilonewton-millimetres
Final_Stiffness_Pct_of_Initial	Final stiffness retention as a percentage of initial stiffness

7 User Preferences and Persistence

7.1 Preferences Storage

User preferences are stored in a JSON file located in the user's home directory, as shown in Equation 75. This persistent storage ensures that user preferences are maintained between application sessions.

$$\text{Path} = \sim/.bbcurve_gui_settings.json \quad (75)$$

7.2 Preference Categories

The preferences are organized into categories that correspond to different aspects of the application's behaviour. Table 24 describes each category and its contents.

Table 24: Preference Categories

Category	Description
Folders	Stores the last input and output folders used by the application
Recent	Maintains lists of recently used input and output folders for quick access
Geometry	Saves the window size and position for consistent user experience
Plot Style	Stores the default plot style preference (matplotlib style)
Line Width	Saves the default line width for plots
Marker Size	Stores the default marker size for plots
Display	Saves the state of display toggles (grid, legend, markers)
Export DPI	Stores the default export resolution for plots

7.3 Preference JSON Schema

The complete JSON schema for the preferences file is shown below. This schema defines all available preferences and their data types.

```
{
  "last_input_dir": "string",
  "last_output_dir": "string",
  "recent_input_dirs": ["string"],
  "recent_output_dirs": ["string"],
  "window_geometry": "string",
  "plot_style": "string",
  "line_width": "number",
  "marker_size": "number",
  "show_grid": "boolean",
  "show_legend": "boolean",
  "show_markers": "boolean",
  "crosshair_enabled": "boolean",
```

```

"export_dpi": "integer",
"max_recent": "integer"
}

```

7.4 Keyboard Shortcuts

The application supports a comprehensive set of keyboard shortcuts for efficient operation. Table 25 lists all available shortcuts and their corresponding actions.

Table 25: Keyboard Shortcuts

Shortcut	Action
Ctrl+O	Open the input folder selection dialog
Ctrl+Shift+O	Open the output folder selection dialog
Ctrl+R	Start processing all files in the input folder
Ctrl+S	Save the current plot to a file
Ctrl+E	Export the current plot with format and DPI options
Ctrl+C	Copy the current plot image to the clipboard
Ctrl+G	Toggle the plot grid on and off
Ctrl+L	Toggle the plot legend on and off
Ctrl+0	Reset the plot view to the default zoom level
Ctrl+F	Focus the specimen search box
F5	Refresh the current plot display
Left	Navigate to the previous specimen
Right	Navigate to the next specimen
Ctrl+,	Open the preferences dialog
Ctrl+Q	Exit the application

8 Complete Equations Reference

This chapter provides a comprehensive listing of all equations implemented in the application, organized by category with sequential numbering for easy reference.

8.1 Data Processing Equations

The following equations govern the initial data processing and validation stages of the analysis. Equation E1 defines the displacement and force arrays that form the fundamental data structure for all subsequent calculations.

$$D = \{d_i \in \mathbb{R} \mid i = 1, \dots, N\}, \quad F = \{f_i \in \mathbb{R} \mid i = 1, \dots, N\} \quad (\text{E1})$$

Equation E2 computes the displacement gradient, which is essential for determining the direction of motion and identifying turning points in the hysteresis curve.

$$\Delta D_i = D_{i+1} - D_i, \quad i = 1, \dots, N - 1 \quad (\text{E2})$$

Equation E3 determines the sign of each gradient, providing a discrete representation of the direction of motion.

$$S_i = \text{sign}(\Delta D_i) = \begin{cases} 1, & \Delta D_i > 0 \\ 0, & \Delta D_i = 0 \\ -1, & \Delta D_i < 0 \end{cases} \quad (\text{E3})$$

Equation E4 identifies turning points where the direction of motion changes, which are critical for extracting the backbone curve and identifying cycles.

$$T_i = \begin{cases} 1, & S_i \neq S_{i-1} \\ 0, & \text{otherwise} \end{cases} \quad (\text{E4})$$

8.2 Envelope Extraction Equations

The following equations govern the extraction of the monotonic envelope, which forms the basis of the backbone curve. Equation E5 defines the clustering tolerance used to group closely spaced points.

$$\tau = 0.02 \times \max(|D|) \quad (\text{E5})$$

Equation E6 identifies the point with the maximum absolute force within each cluster, which is retained as the envelope point.

$$(\bar{D}_j, \bar{F}_j) = \arg \max_{i \in \text{cluster}_j} |F_i| \quad (\text{E6})$$

Equation E7 defines the monotonic envelope by retaining only those points where the absolute displacement strictly increases.

$$\text{Envelope} = \{(\bar{D}_j, \bar{F}_j) \mid \bar{D}_j > \bar{D}_{j-1}\} \quad (\text{E7})$$

8.3 Backbone Curve Equations

The following equations construct the complete backbone curve from the positive and negative branches. Equation E8 separates the data into positive and negative branches.

$$P = \{(D_i, F_i) \mid D_i > 0\}, \quad N = \{(D_i, F_i) \mid D_i < 0\} \quad (\text{E8})$$

Equation E9 extracts the monotonic envelope of the positive branch, while Equation E10 extracts the envelope of the negative branch.

$$(P_x, P_y) = \text{monotonic_envelope}(P_x, P_y) \quad (\text{E9})$$

$$(N_x, N_y) = \text{monotonic_envelope}(-N_x, -N_y) \quad (\text{E10})$$

Equation E11 combines the negative branch, origin, and positive branch to form the complete backbone curve.

$$B = (N_x, N_y) \cup (0, 0) \cup (P_x, P_y) \quad (\text{E11})$$

8.4 Mirroring Equations

Equation E12 mirrors the negative branch onto the positive quadrant, enabling direct comparison with the positive branch.

$$M_x = -\text{reverse}(N_x), \quad M_y = -\text{reverse}(N_y) \quad (\text{E12})$$

8.5 Average Branch Equations

The following equations compute the average branch, which is used for bilinear idealization. Equation E13 creates a common grid from the union of both branch displacement values.

$$G = \text{unique}(P_x \cup M_x) \quad (\text{E13})$$

Equation E14 interpolates both branches onto the common grid, while Equation E15 computes their average.

$$P_y^{(G)} = \text{interp}(G, P_x, P_y), \quad M_y^{(G)} = \text{interp}(G, M_x, M_y) \quad (\text{E14})$$

$$A_y^{(G)} = \frac{P_y^{(G)} + M_y^{(G)}}{2} \quad (\text{E15})$$

8.6 Peak Point Equations

The following equations identify the peak point of the backbone curve. Equation E16 finds the index of the maximum force, while Equation E17 extracts the corresponding force and displacement values.

$$i_{\max} = \arg \max y \quad (\text{E16})$$

$$F_{\max} = y_{i_{\max}}, \quad D_{\max} = x_{i_{\max}} \quad (\text{E17})$$

8.7 Ultimate Point Equations

The following equations identify the ultimate point according to the ASTM E2126 standard. Equation E18 defines the ultimate force as 80% of the peak force.

$$F_u = 0.8F_{\max} \quad (\text{E18})$$

Equation E19 interpolates the ultimate displacement from the data points bracketing the 80% force level.

$$D_u = x_i + \frac{F_u - y_i}{y_{i+1} - y_i}(x_{i+1} - x_i) \quad (\text{E19})$$

8.8 Stiffness Equations

The following equations compute the initial stiffness of the backbone curve. Equation E20 defines the initial stiffness as the secant stiffness through 40% of the peak force.

$$K_e = \frac{0.4F_{\max}}{D_{0.4}} \quad (\text{E20})$$

Equation E21 interpolates the displacement at 40% of the peak force.

$$D_{0.4} = x_i + \frac{0.4F_{\max} - y_i}{y_{i+1} - y_i}(x_{i+1} - x_i) \quad (\text{E21})$$

8.9 Energy Equations

Equation E22 computes the area under the backbone curve up to the ultimate point using the trapezoidal rule.

$$E = \int_0^{D_u} y(x) dx \approx \sum_{i=1}^{n-1} \frac{y_i + y_{i+1}}{2} \Delta x_i \quad (\text{E22})$$

8.10 EEEP Bilinear Idealization Equations

The following equations implement the EEEP bilinear idealization. Equation E23 computes the yield force using the equal energy principle.

$$F_y = K_e D_u - \sqrt{(K_e D_u)^2 - 2K_e E} \quad (\text{E23})$$

Equation E24 computes the yield displacement from the yield force and initial stiffness.

$$D_y = \frac{F_y}{K_e} \quad (\text{E24})$$

Equation E25 computes the displacement ductility ratio.

$$\mu = \frac{D_u}{D_y} \quad (\text{E25})$$

8.11 Cycle Identification Equations

The following equations identify individual cycles in the hysteresis data. Equation E26 identifies positive displacement peaks from the turning points.

$$P_{\text{peak}} = \{i \mid T_i = 1 \text{ and } D_i > 0\} \quad (\text{E26})$$

Equation E27 defines each cycle as the segment of data between consecutive positive peaks.

$$\text{Cycle}_j = \{(D_i, F_i) \mid i_{\text{peak},j} \leq i \leq i_{\text{peak},j+1}\} \quad (\text{E27})$$

8.12 Cycle Metrics Equations

The following equations compute metrics for each identified cycle. Equation E28 computes the secant stiffness for each cycle.

$$K_j = \frac{F_j^{(+)} - F_j^{(-)}}{D_j^{(+)} - D_j^{(-)}} \quad (\text{E28})$$

Equation E29 computes the stiffness retention as a percentage of the initial stiffness.

$$K_{\text{ret},j} = \frac{K_j}{K_1} \times 100\% \quad (\text{E29})$$

Equation E30 computes the energy dissipated in each cycle using the shoelace formula.

$$E_j = \frac{1}{2} \left| \sum_{k=1}^{N_j} (D_k F_{k+1} - D_{k+1} F_k) \right| \quad (\text{E30})$$

Equation E31 computes the cumulative energy dissipation up to each cycle.

$$E_{\text{cum},j} = \sum_{k=1}^j E_k \quad (\text{E31})$$

8.13 Plot Generation Equations

The following equations define the data plotted for each plot type. Equation E32 defines the backbone curve plot data.

$$\text{Backbone Plot: } (D_i, F_i) \in B \quad (\text{E32})$$

Equation E33 defines the positive/negative/average plot data.

$$\text{PNA Plot: } (G_i, P_y^{(G)}), (G_i, M_y^{(G)}), (G_i, A_y^{(G)}) \quad (\text{E33})$$

Equation E34 defines the bilinear idealization plot data.

$$\text{Bilinear Plot: } (D_y, F_y), (D_u, F_y) \quad (\text{E34})$$

Equation E35 defines the stiffness degradation plot data.

$$\text{Stiffness Plot: } (j, K_j) \text{ for } j = 1, \dots, n \quad (\text{E35})$$

Equation E36 defines the energy dissipation plot data.

$$\text{Energy Plot: } (j, E_j) \text{ for } j = 1, \dots, n \quad (\text{E36})$$

9 Usage Guide

9.1 Installation

9.1.1 Python Installation

Before installing the application, ensure that Python 3.7 or higher is installed on your system. The required packages can be installed using the pip package manager, as shown in the code listing below:

```
pip install numpy pandas matplotlib
```

9.1.2 Download Application

Download the `BBCurve.py` file from the developer's GitHub repository or other distribution source and place it in a suitable directory on your system.

9.1.3 Run Application

The application can be launched from the command line using Python, as shown below:

```
python BBCurve.py
```

9.2 Data Preparation

9.2.1 File Format

The input data must be prepared in the format described in Table 26. Following this format ensures that the application can correctly parse and process the data.

Table 26: Input File Format Specification

Property	Description
File Extension	.txt (plain text file)
Delimiter	Tab character or any whitespace (space, tab)
Columns	First column: Displacement (mm); Second column: Force (kN)
Header	First row is skipped (can contain column labels)
Data Format	Numeric values with decimal point (e.g., -6.023, 18.157)

9.2.2 Example Input

A typical input file would look like the example shown below, where each row contains the displacement and force values for a single data point:

```
Displacement Force
-6.023 -18.157
-5.978 -17.892
-5.893 -17.865
-5.723 -17.823
-5.511 -17.802
...
```

9.3 Processing Workflow

The processing workflow consists of four main steps, which are summarized in Table 27. Each step is described in detail below.

Table 27: Processing Workflow Steps

Step	Action
1	Select the input folder containing the TXT files to be processed
2	Select the output folder where results will be saved
3	Start the processing by clicking the Process All Files button or pressing Ctrl+R
4	Monitor the processing progress in the Processing Log tab

9.3.1 Step 1: Select Input Folder

Click the **Choose Input Folder** button in the folder selection panel. A file dialog will appear, allowing you to navigate to and select the directory containing your TXT files. The selected path will be displayed in the input folder text field.

9.3.2 Step 2: Select Output Folder

Click the **Choose Output Folder** button to select the directory where all results (plots, CSV files, and summary) will be saved. The application will automatically create the necessary subdirectories within this folder.

9.3.3 Step 3: Process Files

Click the **Process All TXT Files** button or press **Ctrl+R** to start the processing. The application will process all TXT files in the input folder sequentially, displaying progress information in the Processing Log tab.

9.3.4 Step 4: Monitor Progress

The Processing Log tab provides real-time feedback on the processing progress. Each file is reported with either an OK (green) or FAIL (red) status, and the summary statistics are displayed at the end of the processing run.

9.4 Result Review

9.4.1 Plot Viewer

The Plot Viewer tab allows you to interactively explore the results for each specimen. Use the specimen dropdown menu or the Previous/Next buttons to navigate between specimens. Select different plot types from the dropdown menu to view various aspects of the cyclic behaviour. Use the zoom, pan, and inspection tools for detailed analysis.

9.4.2 Engineering Information

The Engineering Information panel displays key parameters for the selected specimen. These parameters update automatically when you switch specimens or plot types. Click on the plot to inspect individual data points, and the coordinates will be displayed in the panel.

9.4.3 Summary Table

The Summary Table tab provides a comprehensive overview of all processed specimens. You can sort the table by clicking on column headers, filter rows using the search box, and highlight maximum and minimum values using the highlight dropdown menu. Double-clicking on a row will switch to the Plot Viewer tab and display the corresponding specimen.

9.5 Export Results

The application provides several options for exporting results, as summarized in Table 28.

Table 28: Export Options Summary

Action	Description
Save Plot	Save the current plot to a file (Ctrl+S)
Export Plot	Export the current plot with format and DPI options (Ctrl+E)
Copy Plot	Copy the current plot image to the clipboard (Ctrl+C)
Export Log	Export the processing log to a text file
Export Summary	Export the summary table to a CSV file

9.5.1 Export Plot

Click the **Save Plot** button or press **Ctrl+S** to save the current plot. The default format is PNG. Use the **Export** button or **Ctrl+E** to access additional format options including SVG and PDF, with adjustable DPI settings for raster formats.

9.5.2 Copy Plot

Click the **Copy to Clipboard** button or press **Ctrl+C** to copy the current plot image to the system clipboard. This is useful for quickly pasting plots into other applications.

9.5.3 Export Log

Use the **Export Log** button in the Processing Log tab to save the complete processing log to a text file. This is useful for record-keeping and troubleshooting.

9.5.4 Export Summary

Use the **Export Summary Table** option from the File menu or the export buttons in the Summary Table tab to save the summary data to a CSV file. You can choose to export all rows or only the selected rows.

10 Conclusion

10.1 Summary

The **Backbone Curve & Bilinear Idealization Tool** provides a comprehensive, professional-grade solution for analyzing cyclic hysteresis data according to the ASTM E2126 EEEP method. The application successfully combines robust mathematical processing with an intuitive user interface to deliver a powerful analysis tool for structural engineers and researchers.

The key strengths of the application include:

- **Robust Mathematical Processing:** Full implementation of all EEEP method calculations with proper handling of edge cases and data quality issues
- **Professional GUI:** ETABS/SAP2000-style engineering workstation interface with intuitive navigation and comprehensive controls
- **Interactive Visualization:** High-quality plots with full user interaction including zoom, pan, and data inspection
- **Data Management:** Comprehensive CSV output and summary generation with sorting, filtering, and highlighting capabilities
- **User Experience:** Persistent settings, keyboard shortcuts, and intuitive navigation for enhanced productivity
- **Transparency:** Real-time processing log with detailed feedback and color-coded status indicators

10.2 Applications

The tool is suitable for a wide range of applications in structural engineering and research, as summarized in Table 29.

10.3 Future Developments

Potential future enhancements to the application include:

- Support for additional cyclic analysis methods beyond the EEEP approach
- Integration with machine learning algorithms for data classification and prediction

Table 29: Application Areas and Use Cases

Area	Description of Applications
Research	Parametric studies of structural behaviour under cyclic loading, development of constitutive models
Education	Teaching structural dynamics, seismic analysis, and experimental methods
Engineering Practice	Performance-based seismic design, capacity assessment of existing structures
Industry	Quality control and acceptance testing of structural components and materials

- Enhanced statistical analysis capabilities for large datasets
- Automated report generation with customizable templates
- Support for additional file formats including Excel and other standard formats
- API development for integration with other analysis tools and workflows
- Cloud-based processing for very large datasets

10.4 Final Remarks

This application represents a significant contribution to the field of structural engineering by providing a standardized, automated, and user-friendly tool for cyclic data analysis. The implementation follows established engineering standards while incorporating modern software design principles to ensure reliability, performance, and usability. The extensive documentation provided in this manual serves as both a user guide and a technical reference for the underlying algorithms and mathematics.

The developer encourages users to explore the full capabilities of the tool and welcomes feedback for future improvements. The application is designed to be extensible and can be adapted to meet specific user requirements as needed.

A Glossary of Technical Terms

Table 30: Glossary of Technical Terms Used in This Document

Term	Definition
Backbone Curve	The envelope of the hysteresis loops, representing the maximum force at each displacement level
Bilinear Idealization	A simplified elastic-perfectly-plastic representation of the backbone curve used for design
Ductility	The ratio of ultimate displacement to yield displacement, $\mu = D_u/D_y$
EEEP	Equivalent Energy Elastic-Plastic method, a standard approach for characterizing cyclic behaviour
Hysteresis	The dependence of the force-displacement relationship on the loading history
Hysteresis Loop	A closed path in the force-displacement space representing one complete loading cycle

Term	Definition
Initial Stiffness	The secant stiffness through 40% of the peak force on the ascending branch
Monotonic Envelope	The curve through the local maxima of the response, representing the outermost bounds
Pinching	The narrowing of hysteresis loops due to crack opening and closing mechanisms
Secant Stiffness	The stiffness defined by the slope of the line connecting two points on a hysteresis loop
Stiffness Degradation	The progressive loss of stiffness with increasing number of loading cycles
Ultimate Point	The point at which the force drops to 80% of the peak force after the peak
Yield Point	The idealized point representing the transition from elastic to perfectly plastic behaviour